

March *not-so* Madness

Needle in a Haystack Final Project

Team 41

Barak Schwartz	327359592	baraksc	Itay.maman@mail.huji.ac.il
Itay Maman	206554677	itay_maman	Barak.schwartz@mail.huji.ac.il
Itamar Kroitoru	209173533	itamar_35	Itamar.kroitoru@mail.huji.ac.il

Probabilistic Prediction of NCAA Basketball Games



Domain: NCAA Division I Men's Basketball

The domain of our project is NCAA Division I men's basketball. Division I includes more than 350 college teams from across the United States, divided into over 30 conferences. Each team plays roughly 30 games during a season, followed by conference tournaments and, for qualifying teams, the NCAA Tournament ("March Madness"). The large number of teams and games produces a rich statistical environment. Teams differ considerably in their style and level of play, which can be reflected in statistics such as scoring, shooting percentages, rebounds, assists, turnovers, and many others. These differences, together with factors such as the opponent and game location, make college basketball an interesting domain for comparing teams and attempting to predict the outcomes of future games.

Data Description

Our data comes from Kaggle's *March Machine Learning Mania 2026* dataset. Although the full dataset contains many different files, we focused on the data directly relevant to our prediction problem.

Our main source was "MRegularSeasonDetailedResults.csv", which contains 124,529 NCAA Division I men's regular-season games from the 2003 through 2026 seasons. The file is 11.81 MB and contains 34 fields for each game. These include the two teams, final score, game location, number of overtimes, and detailed box-score statistics for both teams: field goals, three-pointers, free throws, offensive and defensive rebounds, assists, turnovers, steals, blocks, and personal fouls.

We also used "MTteams.csv", which contains 381 teams and is approximately 0.01 MB. Its four fields contain each team's ID and name, as well as the first and last seasons in which it competed in Division I. We used this file to connect the numerical team IDs in the game data to their actual team names.

The data is complete, with no missing values in either file, and we did not identify any major data-quality issues or obvious biases that required correction.

Chosen Problem: Single Game Prediction

Our goal is simple: given two NCAA Division I teams about to play each other, can we use only the information available before the game to estimate the probability that each team will win?

Basketball is unpredictable, and we are not arrogant enough to expect a model to predict every game correctly. Instead, our hypothesis is that the history of both teams up to that point – including their performance, box-score statistics, recent form, opponent strength, and game location – contains enough information to produce a meaningful probability of the game's outcome.

Solution - Overview:

We treated each historical game as a supervised-learning example, reconstructing both teams' information immediately before the game and using the winner as the label. We then engineered matchup features, performed feature selection, and compared several supervised-learning models and an ensemble to predict win probabilities.

Dataset Construction

Our first challenge came from the structure of the original Kaggle data. Each row records a completed game, while our prediction requires describing both teams using only information available before that game. We therefore processed every season chronologically and created a TeamState object for each team that accumulated its performance as games were played.

Each TeamState tracked games played, wins and losses, points scored and allowed, shooting, rebounds, assists, turnovers, and other box-score statistics, from which we calculated features such as win percentage, points per game and shooting percentages. To account for strength of schedule, we also recorded information about each opponent's state when a game was played, producing features such as opponents' average win percentage and point differential.

Before each game, the current TeamState of both teams was used to create a pre-game observation. Only afterward were their states updated with the current game, ensuring that no information from the game itself could leak into its prediction.

For example, suppose Duke plays Kansas partway through the 2021 season. Immediately before the game, their states might produce:

Feature	Team1: Duke	Team2: Kansas
Games Played	15	16
Win Percentage	0.73	0.81
Points Per Game	78.2	81.4
...	...	...
Opponent Win Percentage	0.62	0.51
Winner (label)	1	0

This gave us our **raw dataset**, where every historical game was represented by two separate sets of pre-game statistics: one for Team 1 and one for Team 2. Games in which either team had not yet played that season were excluded, since no season-specific prior information was available. The resulting raw dataset had a shape of **(119,412, 131)**.

Before performing feature analysis or model development, we separated the data chronologically. Games from **2003–2025 formed the training set**, while the entire **2026 season was held out as the test set**. This ensured that information from the test season could not influence feature-engineering or feature-selection decisions.

After feature engineering, corresponding Team 1 and Team 2 features were replaced by their difference (**Team 1 – Team 2**). For example, team_1_ppg and team_2_ppg became ppg_diff, so each row represented Team 1's statistical advantages and disadvantages relative to Team 2.

Feature	Differential (Team 1 – Team 2)
Games Played	-1
Win Percentage	-0.08
Points Per Game	-3.2
...	...
Opponent Win Percentage	0.11
Winner (label)	1

Finally, we **symmetrized the training set** so that the arbitrary Team 1/Team 2 ordering could not influence the model. Each training matchup was represented twice: once normally and once with the

teams reversed. In the reversed row, differential features changed sign and the outcome label was reversed. The held-out **2026 test set was not symmetrized**, so that each real game appeared exactly once during final evaluation. This allowed the model to learn relative team differences while keeping the test set representative of actual games.

Feature Engineering

In addition to the statistics constructed during preprocessing, we created several **interaction features** to capture relationships that individual features might miss. For example, a team's historical home win percentage is most relevant when the current game is actually being played at home, so we constructed location-specific strength features that activate only for the relevant game location.

We also combined win percentage with the number of games already played. A win percentage based on only a few games is less informative than the same percentage later in the season, so we created a season-progress-adjusted win-rate feature that moves gradually from a neutral value toward the observed win percentage as more games are played.

These transformations were defined using the **training data and basketball context**, and the exact same deterministic transformations were then applied to the held-out 2026 data. The test outcomes were never used during this process.

Feature Selection

After preprocessing and feature engineering, we were left with a large number of candidate features. Keeping all of them was not necessarily beneficial: some contained nearly identical information, while others added little predictive value. All feature-analysis and selection decisions were therefore performed using the **2003–2025 training data only**, with the 2026 season remaining untouched until final model evaluation.

We first examined **Pearson correlations** between training features to identify highly redundant pairs. Pairs with an absolute correlation above **0.90** were flagged for manual inspection. For example, a team's average opponent win percentage over its last ten games had a correlation of approximately **0.992** with its season-long opponent win percentage, indicating that the two features contained almost the same information. Based on this analysis, selected redundant variables were removed before further processing.

We then evaluated the remaining features using three different feature-selection methods, each approaching feature importance in a different way:

Method	How it works	What it captures
Lasso (L1)	Fits a linear model while penalizing large coefficients. Features whose coefficients shrink to 0 are removed.	Linear relationships between individual features and the target.
RFE	Repeatedly fits a model, ranks features by importance/coefficients, and removes the weakest features until the desired number remains.	Model-dependent predictive importance ; can capture more than simple linear relevance depending on the estimator used.
Tree Importance	Trains a decision tree/forest and ranks features by how much they reduce impurity when used for splits.	Nonlinear relationships and interactions between features.

Lasso also provided a useful way to visualize which features remained important as the amount of regularization increased. As the penalty becomes stronger, less useful features are pushed to zero earlier, while stronger features remain in the model for longer.

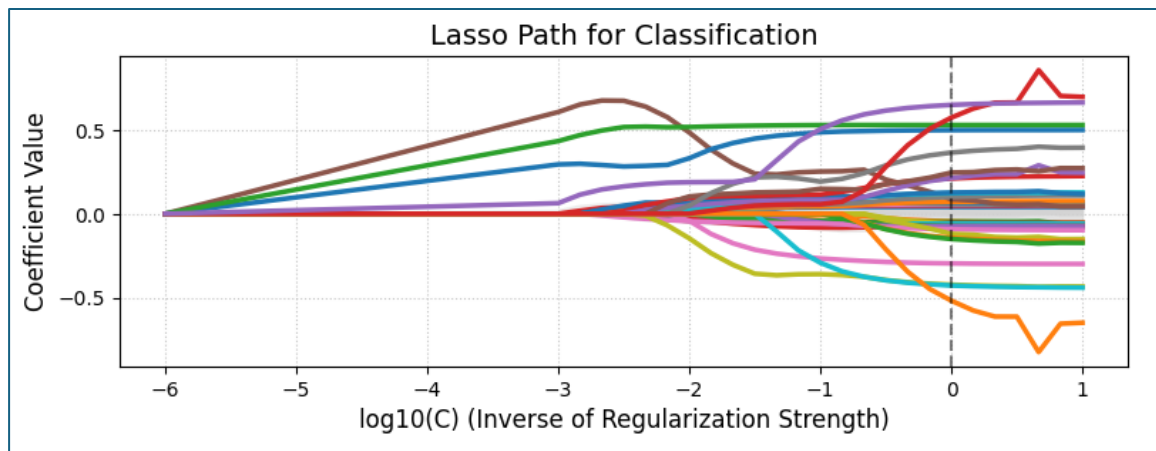


Figure 1: Lasso paths across regularization strengths. Each line represents one feature's coefficient; moving left corresponds to stronger regularization, progressively shrinking coefficients toward zero. The dashed line marks $C=1$.

Finally, rather than choosing the number of features arbitrarily, we used **backward elimination**, progressively removing the weakest features and evaluating predictive performance after each step. This allowed us to reduce the size of the feature set while checking that we were not sacrificing useful predictive information.

Based on this process, we selected a final set of **35 differential features** for model training.

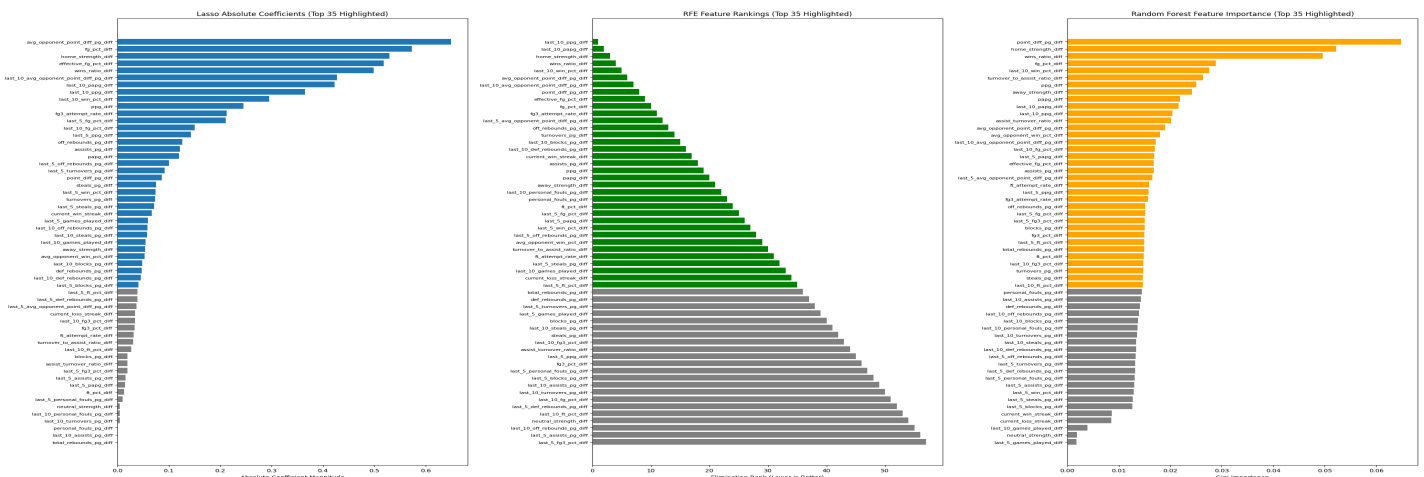


Figure 2: Feature rankings from three complementary selection methods - Lasso, RFE, and Random Forest - with the top 35 highlighted to compare the features prioritized by each approach.

Model Fitting

With the final feature set in place, the next step was to train models that could turn the pre-game differential features into win probabilities. Rather than committing to a single type of model, we tested several supervised-learning approaches with different levels of complexity:

- **Logistic Regression** provided a simple linear and naturally probabilistic model. The input features were standardized before fitting, and we used L2 regularization with $C=0.1$, giving relatively strong regularization to reduce overfitting.
- **LightGBM** was used to capture nonlinear relationships and feature interactions through gradient-boosted decision trees. We used 500 trees with a learning rate of 0.02 and maximum depth of 5, together with row and feature subsampling and both L1 and L2 regularization.
- **XGBoost** provided a second gradient-boosting approach. We used 800 trees with a smaller learning rate of 0.01, maximum depth of 6, and 60% row and feature subsampling. The relatively low learning rate and subsampling were chosen to limit overfitting while allowing the model to learn more complex relationships.
- **Random Forest** combined 500 decision trees, with maximum depth limited to 12 and at least 10 samples required in each leaf. Each split considered only the square root of the available features, encouraging diversity between trees.
- **K-Nearest Neighbors (KNN)** predicted games based on similar historical matchups. Because KNN is sensitive to feature scale, the features were standardized first. We used 35 neighbors with distance-based weighting and Euclidean distance.
- **Multi-Layer Perceptron (MLP)** provided a neural-network approach. The network contained two hidden layers of 64 and 32 neurons with ReLU activation. We standardized the inputs, used L2 regularization ($\alpha=0.001$), and enabled early stopping to reduce overfitting.

In addition to evaluating each model individually, we experimented with a **stacking ensemble**. The predicted probabilities from the individual models were used as inputs to a logistic-regression meta-learner, with the idea that models using different learning approaches might capture different patterns and compensate for one another's mistakes.

Evaluation

We split the data chronologically **immediately after raw dataset construction: 2003–2025** for training and **2026** as a fully held-out test season. All feature engineering and feature selection were based only on training data, with the same fixed transformations applied to 2026.

We evaluated models using **accuracy, log loss, and ROC-AUC**. Because the goal is to produce meaningful win probabilities, **log loss** was especially important, as it penalizes confident incorrect predictions.

As a baseline, we used a **50/50 predictor**, corresponding to approximately **0.69 log loss** and **0.5 ROC-AUC**.

Results:

Model	Test Accuracy	Test Log Loss	Test ROC-AUC
Baseline 50/50 Coin Toss	~50%	0.69	0.5
Logistic Regression	70.14%	0.55	0.78
LightGBM	70.70%	0.55	0.79

XGBoost	70.75%	0.55	0.79
Random Forest	70.86%	0.55	0.79
KNN	69.54%	0.57	0.77
MLP	70.27%	0.55	0.78

Despite their different structures, the models produced similar results, with most achieving around **70–71% test accuracy**, **0.55 log loss**, and **0.78–0.79 ROC-AUC**. **Random Forest** was the strongest individual model by accuracy at **70.86%**, while KNN performed worst at **69.54% accuracy** and **0.77 ROC-AUC**. Overall, the limited differences suggest that additional model complexity provided only modest gains.

We also evaluated a **stacking ensemble**, using predictions from the base models as inputs to a Logistic Regression meta-learner. The ensemble achieved **70.95% accuracy**, **0.55 log loss**, and **0.79 ROC-AUC**, only slightly outperforming the individual models.

Results:

Model	Test Accuracy	Log Loss	Test ROC-AUC
Ensemble	70.95%	0.55	0.79

The models’ predictions were highly correlated, helping explain why stacking added little benefit: despite their different architectures, they tended to make similar predictions and similar mistakes.

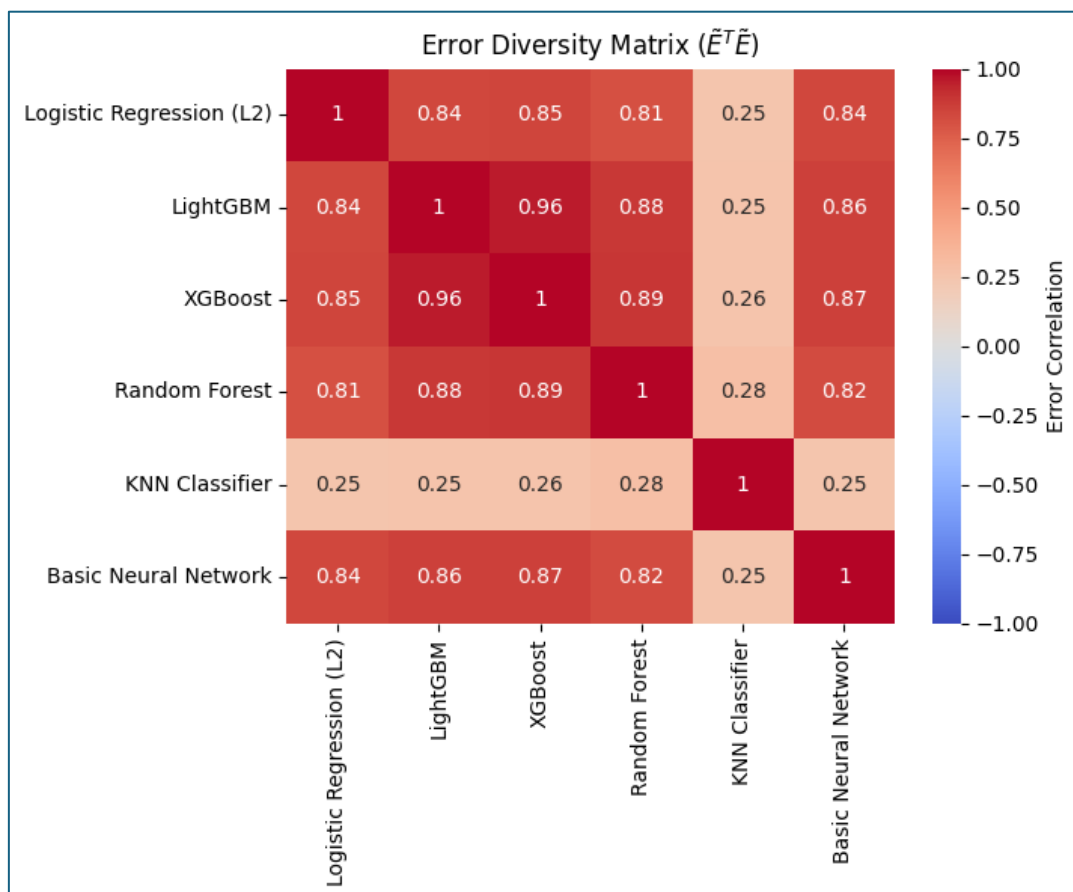


Figure 3: Error correlation across the six models. Most models show strongly correlated errors, indicating that they tend to make mistakes on the same games and explaining the limited benefit of stacking. KNN is the main exception, showing substantially lower error correlation with the other models.

Given these small differences, we selected **Logistic Regression as our final model**. Although its accuracy was slightly lower at **70.14%**, it achieved comparable probabilistic performance while being substantially **simpler and more interpretable** than the alternatives.

Real-Life Usage

To visualize how the final model performed on unseen data, we examined its predictions throughout the held-out 2026 season. The figure below shows the number of correct and incorrect predictions for each day of the season. The model maintained strong performance across the season, achieving **71.1% overall accuracy**, while the varying bar heights also reflect the substantial differences in the number of games played on different days.

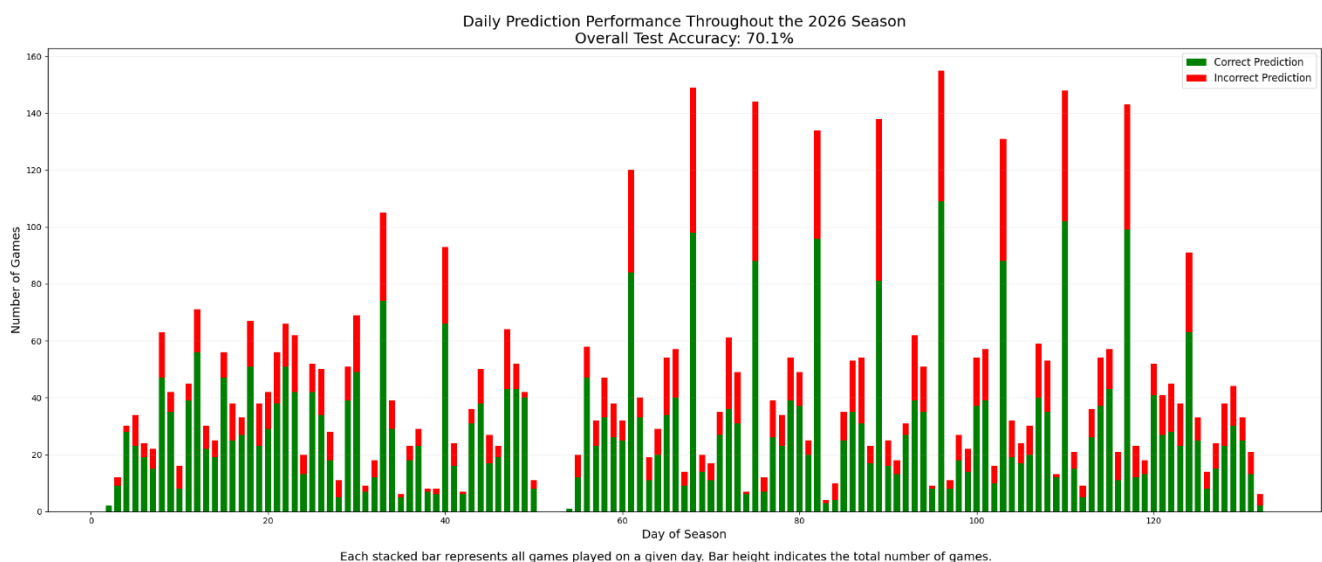


Figure 4: Daily prediction performance throughout the held-out 2026 season. Correct predictions consistently outnumber incorrect predictions across the season, resulting in 70.1% overall accuracy.

Overall, the figure shows that correct predictions consistently outnumber incorrect ones throughout the test season, including on the busiest game days. The empty gap around days 52-55 corresponds to the NCAA's Christmas and winter-break pause: games stopped after December 23 and resumed on December 27, reflecting the required three-day break for student-athletes.

Interactive Web Application - [GitHub](#) , [App](#)



To make the final model easier to explore, we developed an interactive web application that presents both its predictions and evaluation results. The Match Simulator replays games from the held-out 2026 season. Users can select a day and matchup, inspect both teams' pre-game statistics, view the model's estimated win probabilities, and then reveal the actual result to determine

whether the prediction was correct. Since the model was trained only on earlier seasons, **this demonstrates how it performs on genuinely unseen games.**

The application also includes a What-If Predictor for comparing any two team-season combinations from 2003-2026 while accounting for game location. It displays the predicted winner, win probabilities, confidence level, estimated point spread, and the statistical differences that contributed most to the prediction. A Model Insights page further explains the prediction pipeline, presents the most influential Logistic Regression weights, and visualizes daily performance across the 2026 test season.

Impediments

One of the main challenges was preventing **data leakage**, since each prediction could only use information available beforehand. We addressed this by processing games chronologically and updating each TeamState only after creating the prediction row. We also encountered **redundant features**, addressed through correlation analysis and feature selection. Finally, the ensemble added little because the individual models produced highly correlated predictions, leading us to favor the simpler Logistic Regression model.

Conclusions & Future Work

Our models achieved around **70–71% test accuracy**, substantially outperforming the 50/50 baseline. The similar performance of very different models suggests that greater complexity alone may offer limited gains. Future improvements will likely require better **data and feature engineering** to capture information missing from our current dataset.

A natural next step is to extend the model to forecasting an entire **NCAA Tournament**. Rather than predicting one bracket, we could **simulate the tournament thousands of times** using our estimated game probabilities, producing probabilities for each team to reach each round and ultimately win the championship.